Protobuf Dynamic Transcoder — MVP Spec (Stage 1)

Goal

Given raw `.proto` schema files and protobuf-encoded bytes, decode them into an internal model, then serialize that model to JSON.

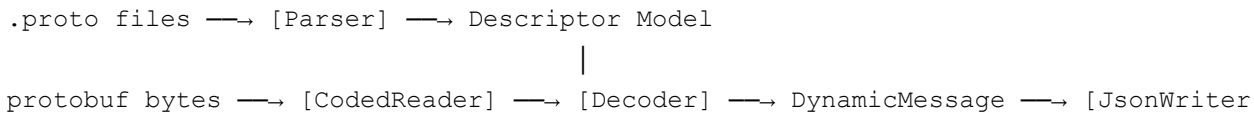
That's it. No encode. No JSON-to-protobuf. Decode only.

Constraints

- No external NuGet packages (no `Google.Protobuf`, no `protobuf-net`)
- No `System.Reflection`, no `Emit`
- AOT compatible (`PublishAot` on .NET 10)
- Only runtime dependency: `System.Text.Json`

Architecture

Three layers:



Layer 1: Proto Parser + Descriptor Model

Parser: Hand-written recursive descent. Parses `.proto` text, produces `FileDescriptorProto` (our own type mirroring `descriptor.proto`).

Proto3 subset to parse (see Proto3 Language Guide):

- `syntax`, `package`, `import`
- `message` (including nested), `enum`, `oneof`, `map<K,V>`, `repeated`
- Scalar types, message/enum references

- `option` (parsed, stored opaquely)
- `reserved`, comments
- `service` / `rpc` (parsed, ignored)

Descriptor model: Resolves cross-references from the raw parse output. Builds lookup tables.

This is a port of Java's `Descriptors.java`, specifically the `FileDescriptor.buildFrom()` method that resolves unresolved type references and the inner classes `Descriptor`, `FieldDescriptor`, `EnumDescriptor`, and `EnumValueDescriptor`.

```
public sealed class DescriptorPool
{
    public static DescriptorPool Build(
        IProtoFileResolver resolver,
        params string[] protoFiles);

    public static DescriptorPool Build(
        params ProtoFile[] schemas);

    public MessageDescriptor FindMessageType(string fullName);
    public EnumDescriptor FindEnumType(string fullName);
}

public readonly record struct ProtoFile(string Name, string Content);

public interface IProtoFileResolver
{
    string? Resolve(string importPath);
}
```

Key resolved types:

```
public sealed class MessageDescriptor
{
    public string FullName { get; }
    public IReadOnlyList<FieldDescriptor> Fields { get; }
    public IReadOnlyList<OneofDescriptor> Oneofs { get; }

    public FieldDescriptor? FindFieldByNumber(int number);
}

public sealed class FieldDescriptor
{
```

```
    public string Name { get; }
    public string JsonName { get; }       // lowerCamelCase
    public int Number { get; }
    public FieldType Type { get; }        // INT32, STRING, MESSAGE, ENUM, BYTES,
    public bool IsRepeated { get; }
    public bool IsMap { get; }
    public bool IsPacked { get; }
    public MessageDescriptor? MessageType { get; }  // resolved ref
    public EnumDescriptor? EnumType { get; }        // resolved ref
    public OneofDescriptor? ContainingOneof { get; }
}

public sealed class EnumDescriptor
{
    public string FullName { get; }
    public EnumValueDescriptor? FindValueByNumber(int number);
}

public sealed class EnumValueDescriptor
{
    public string Name { get; }
    public int Number { get; }
}

public sealed class OneofDescriptor
{
    public string Name { get; }
    public IReadOnlyList<FieldDescriptor> Fields { get; }
}
```

Well-known `.proto` files (`google/protobuf/timestamp.proto` , `wrappers.proto` , `struct.proto` , etc.) are embedded and auto-resolved. The full set is available in the protobuf source tree.

Layer 2: Wire Format Reader

Port of Java's `CodedInputStream.java` as a `ref struct` over `ReadOnlySpan<byte>` .

The wire format is documented in the Encoding Reference. Key concepts: tag = `(field_number << 3) | wire_type` , six wire types (VARINT, I64, LEN, SGROUP, EGROUP, I32), varint encoding for variable-length integers, zigzag encoding for signed types.

```
public ref struct CodedReader
{
    public CodedReader(ReadOnlySpan<byte> buffer);
```

```
    public bool IsAtEnd { get; }

    public uint ReadTag();
    public static int GetFieldNumber(uint tag);
    public static WireType GetWireType(uint tag);

    // Primitives
    public int ReadInt32();
    public long ReadInt64();
    public uint ReadUInt32();
    public ulong ReadUInt64();
    public int ReadSInt32();
    public long ReadSInt64();
    public bool ReadBool();
    public float ReadFloat();
    public double ReadDouble();
    public uint ReadFixed32();
    public ulong ReadFixed64();
    public int ReadSFixed32();
    public long ReadSFixed64();
    public string ReadString();
    public ReadOnlySpan<byte> ReadBytes();
    public int ReadEnum();

    // Returns a sub-reader scoped to the embedded message bytes
    public CodedReader ReadMessage();

    // Skip unknown field
    public void SkipField(uint tag);
}

public enum WireType
{
    Varint = 0,
    Fixed64 = 1,
    LengthDelimited = 2,
    StartGroup = 3,
    EndGroup = 4,
    Fixed32 = 5,
}
```

Java's `CodedInputStream` has multiple internal implementations (array-backed, stream-backed, ByteBuffer-backed). For the MVP we only need the array/span-backed variant, which maps to the `ArrayDecoder` inner class in CodedInputStream.java.

Layer 3: DynamicMessage + JSON Serialization

DynamicMessage — port of Java's `DynamicMessage.java`. Immutable once built, constructed via an internal mutable `Builder` during decode (matching the Java pattern).

```
public sealed class DynamicMessage
{
    public MessageDescriptor Descriptor { get; }

    public object? GetField(FieldDescriptor field);
    public bool HasField(FieldDescriptor field);
    public bool HasOneof(OneofDescriptor oneof);
    public FieldDescriptor? GetOneofFieldDescriptor(OneofDescriptor oneof);

    /// <summary>
    /// Decode protobuf bytes into a DynamicMessage using the given descriptor.
    /// </summary>
    public static DynamicMessage Decode(
        MessageDescriptor descriptor,
        ReadOnlySpan<byte> protobufBytes);
}
```

The `Builder` is used internally during decode and can be used in tests to construct messages programmatically:

```
public sealed class DynamicMessage.Builder
{
    public static Builder NewBuilder(MessageDescriptor type);

    public Builder SetField(FieldDescriptor field, object value);
    public Builder AddRepeatedField(FieldDescriptor field, object value);
    public Builder ClearField(FieldDescriptor field);

    public DynamicMessage Build();
}
```

Field value storage:

Proto type	C# type
int32, sint32, sfixed32	`int`
int64, sint64, sfixed64	`long`
uint32, fixed32	`uint`
uint64, fixed64	`ulong`

float	`float`
double	`double`
bool	`bool`
string	`string`
bytes	`byte[]`
enum	`EnumValueDescriptor`
message	`DynamicMessage`
repeated X	`List<T>`
map<K,V>	`List<DynamicMessage>` (entry messages)

Decode logic — the core loop. This is a port of the decode path in Java's MessageReflection.java `mergeMessageFrom()` and the wiring in AbstractMessage.java `mergeFrom(CodedInputStream)`. The C++ equivalent in wire_format.cc `ParseAndMergePartial()` is actually the clearest reference for the pure decode loop:

```
Decode(descriptor, bytes):
    reader = new CodedReader(bytes)
    builder = Builder.NewBuilder(descriptor)

    while not reader.IsAtEnd:
        tag = reader.ReadTag()
        fieldNumber = GetFieldNumber(tag)
        wireType = GetWireType(tag)
        field = descriptor.FindFieldByNumber(fieldNumber)

        if field is null:
            reader.SkipField(tag)
            continue

        if field.IsPacked and wireType == LengthDelimited:
            subReader = reader.ReadMessage()
            while not subReader.IsAtEnd:
                value = ReadValue(subReader, field)
                builder.AddRepeatedField(field, value)
        else:
            value = ReadValue(reader, field)
            if field.IsRepeated:
```

```
                builder.AddRepeatedField(field, value)
            else:
                builder.SetField(field, value)

    return builder.Build()

ReadValue(reader, field):
    match field.Type:
        INT32    → reader.ReadInt32()
        INT64    → reader.ReadInt64()
        UINT32   → reader.ReadUInt32()
        UINT64   → reader.ReadUInt64()
        SINT32   → reader.ReadSInt32()
        SINT64   → reader.ReadSInt64()
        BOOL     → reader.ReadBool()
        FLOAT    → reader.ReadFloat()
        DOUBLE   → reader.ReadDouble()
        FIXED32  → reader.ReadFixed32()
        FIXED64  → reader.ReadFixed64()
        SFIXED32 → reader.ReadSFixed32()
        SFIXED64 → reader.ReadSFixed64()
        STRING   → reader.ReadString()
        BYTES    → reader.ReadBytes().ToArray()
        ENUM     → field.EnumType.FindValueByNumber(reader.ReadEnum())
        MESSAGE  → DynamicMessage.Decode(field.MessageType, reader.ReadMessage())
```

JSON serialization — port of the `Printer` inner class in Java's `JsonFormat.java`, adapted to use `Utf8JsonWriter` instead of Gson. The JSON mapping is specified in the ProtoJSON Format documentation.

```
public sealed class DynamicMessageJsonSerializer
{
    public DynamicMessageJsonSerializer(JsonSerializeOptions? options = null);

    public void Write(DynamicMessage message, IBufferWriter<byte> output);
    public byte[] Write(DynamicMessage message);
}

public sealed class JsonSerializeOptions
{
    public bool UseProtoFieldNames { get; init; }  // default false (camelCase)
    public bool EmitDefaults { get; init; }        // default false
    public bool EnumsAsIntegers { get; init; }     // default false
}
```

JSON mapping (proto3 canonical, per the ProtoJSON spec):

Proto type	JSON
int32/sint32/sfixed32/uint32/fixed32	number
int64/sint64/sfixed64/uint64/fixed64	string
float/double	number (`"NaN"` , `"Infinity"` , `"-Infinity"` for special values)
bool	boolean
string	string
bytes	base64 string
enum	string name (or int if `EnumsAsIntegers`)
message	object (recurse)
repeated	array
map<K,V>	object (key coerced to string)

Well-known type handlers (checked by `FullName` before generic walk). These are all implemented as special cases in Java's `JsonFormat.Printer` — see the `wellKnownTypePrinters` map and the `printTimestamp` , `printDuration` , `printStruct` , `printValue` , `printListValue` , `printWrapper` , `printAny` , `printFieldMask` methods in `JsonFormat.java` :

Type	JSON
`google.protobuf.Timestamp`	`"2024-01-15T12:00:00Z"`
`google.protobuf.Duration`	`"1.5s"`
`google.protobuf.Struct`	native JSON object
`google.protobuf.Value`	native JSON value
`google.protobuf.ListValue`	native JSON array
`google.protobuf.XxxValue` wrappers	unwrapped value or `null`

`google.protobuf.Any`	`{"@type": "...", ...}` — resolved via `TypeRegistry.java` pattern
`google.protobuf.FieldMask`	`"field1,field2"`
`google.protobuf.Empty`	`{}`

Top-Level API

```
public sealed class ProtoTranscoder
{
    public static ProtoTranscoder Create(
        IProtoFileResolver resolver,
        params string[] protoFiles);

    public static ProtoTranscoder Create(
        params ProtoFile[] schemas);

    public DescriptorPool Pool { get; }

    /// <summary>Decode protobuf bytes to the internal model.</summary>
    public DynamicMessage Decode(
        string messageTypeName,
        ReadOnlySpan<byte> protobufBytes);

    /// <summary>Decode protobuf bytes straight to JSON.</summary>
    public byte[] ToJson(
        string messageTypeName,
        ReadOnlySpan<byte> protobufBytes,
        JsonSerializeOptions? options = null);

    /// <summary>Streaming JSON output.</summary>
    public void ToJson(
        string messageTypeName,
        ReadOnlySpan<byte> protobufBytes,
        IBufferWriter<byte> jsonOutput,
        JsonSerializeOptions? options = null);
}
```

Error Handling

- `ProtoParseException` — bad `.proto` syntax (file, line, column, message)
- `DecodeException` — wire format errors (malformed varint, truncated data, wire type mismatch)
- Unknown fields during decode: silently skipped (standard protobuf behavior, see Encoding Reference on unknown fields)
- Unknown message type name: `ArgumentException`

Implementation Order

1. **CodedReader** — wire format reading over `Span<byte>`. Port the `ArrayDecoder` inner class from `CodedInputStream.java`. Test against known protobuf byte sequences.
2. **Descriptor model** — `MessageDescriptor`, `FieldDescriptor`, `EnumDescriptor` with lookup tables. Port the resolution logic from `Descriptors.java`. Construct programmatically in tests (no parser yet).
3. **DynamicMessage.Decode** — the core decode loop. Port from `MessageReflection.java` `mergeMessageFrom()` and `DynamicMessage.java` `Builder`. Feed it a descriptor + protobuf bytes, get a `DynamicMessage` back. Test round-trip against Java/protoc output.
4. **DynamicMessageJsonSerializer** — walk `DynamicMessage` → JSON. Port the `Printer` from `JsonFormat.java`. Test against proto3 canonical JSON vectors.
5. **Proto parser** — `.proto` text → `FileDescriptorProto` → resolve into `DescriptorPool`. Test by parsing well-known protos from the protobuf source tree.
6. **Wire it up** — `ProtoTranscoder` facade.

Steps 1–4 give us a working decode pipeline with programmatic descriptors. Step 5 adds `.proto` file support. Step 6 is glue.

Out of Scope

- Encode (JSON → protobuf, DynamicMessage → bytes)
- Proto2, Editions 2024 (Stage 2)
- Extensions (Stage 2)

- gRPC / HTTP transcoding
- Streaming multi-message decode

References

Official Protobuf Specifications

- Wire Format Encoding Reference — tag encoding, varint, zigzag, wire types, groups, packed repeated
- Proto3 Language Guide — grammar, scalar types, maps, oneofs, options
- ProtoJSON Format — canonical JSON mapping, well-known type representations
- `descriptor.proto` — the proto meta-schema (FileDescriptorProto, DescriptorProto, FieldDescriptorProto, etc.)
- Well-known `.proto` files — timestamp.proto, wrappers.proto, struct.proto, any.proto, etc.

Java Source Code (primary port reference)

- `DynamicMessage.java` — **port target for `DynamicMessage`**. Immutable message + mutable `Builder`. ~740 lines.
- `Descriptors.java` — **port target for descriptor model**. `FileDescriptor`, `Descriptor`, `FieldDescriptor`, `EnumDescriptor`, `OneofDescriptor`. Cross-reference resolution in `buildFrom()`.
- `CodedInputStream.java` — **port target for `CodedReader`**. Focus on the `ArrayDecoder` inner class. Varint reading, fixed reads, length-delimited, skip logic.
- `JsonFormat.java` — **port target for JSON serialization**. `Printer` inner class. Well-known type handlers in `wellKnownTypePrinters` map.
- `MessageReflection.java` — **port target for decode loop**. `mergeMessageFrom()` contains the tag→field→dispatch logic.
- `AbstractMessage.java` — wires `mergeFrom(CodedInputStream)` to `MessageReflection.mergeMessageFrom()`.
- `TypeRegistry.java` — type registry for resolving `google.protobuf.Any` type URLs.
- `UnknownFieldSet.java` — unknown field preservation during decode.

C++ Source Code (supplementary — clearer decode loop)

- `wire_format.cc` — `ParseAndMergePartial()` is the clearest single-function implementation of the full decode loop: read tag → find field by number → dispatch by wire type → recurse for messages.
- `dynamic_message.h` — C++ `DynamicMessageFactory`, useful for understanding the factory/cache pattern.